

OWASP Juice Shop Security Assessment

An in-depth review of scanning, SQL Injection, and business logic weaknesses.

Made by

- Mark Mekhael Adly
- Hassam Amgad Hassa
- Shahd Ahmed Mohamad
- Mohamed Mahmoud Ali





Target

Scope: Target Environment



Target Environments

- <http://demo.owasp-juice.shop>
- <http://preview.owasp-juice.shop>

Testing Methodology

Black-box approach, no internal access or prior knowledge

Assessment Coverage

Critical client-side and server-side components, simulating external attacker behavior



Standards Referenced



Referenced Standards

OWASP Top 10 framework for vulnerabilities



Threat Modeling

Aligned with real-world attacker tactics and patterns



Objectives

1

Persistence Testing

Assess app resistance to sustained unauthorized access

2

Vulnerability Identification

Detect and analyze critical security flaws across the application

3

Real-World Exploitation

Simulate attacker behavior in realistic, production-like scenarios

4

Security Hardening

Recommend improvements to access control and session management

Reconnaissance



Comprehensive Pentest Report: OWASP Juice Shop Assessment

This presentation details the comprehensive penetration testing activities conducted on the OWASP Juice Shop environment using Acunetix vulnerability scanner. The report is based on a thorough assessment exported directly from Acunetix, providing detailed insights throughout the engagement.

Our focus spans initial scanning and reconnaissance to identify vulnerabilities, exploitation of SQL injection flaws for persistence, and discovery of critical additional security weaknesses. We provide actionable remediation recommendations to strengthen the security posture of the target applications.

Findings highlight significant attack surfaces including exposed backup files, configuration missteps, outdated components, and critical authentication bypasses. Applying these detailed insights enables prioritized risk mitigation and improves defenses against potential adversaries.

Part 1: Scanning & Reconnaissance Insights

Vulnerability Scanning

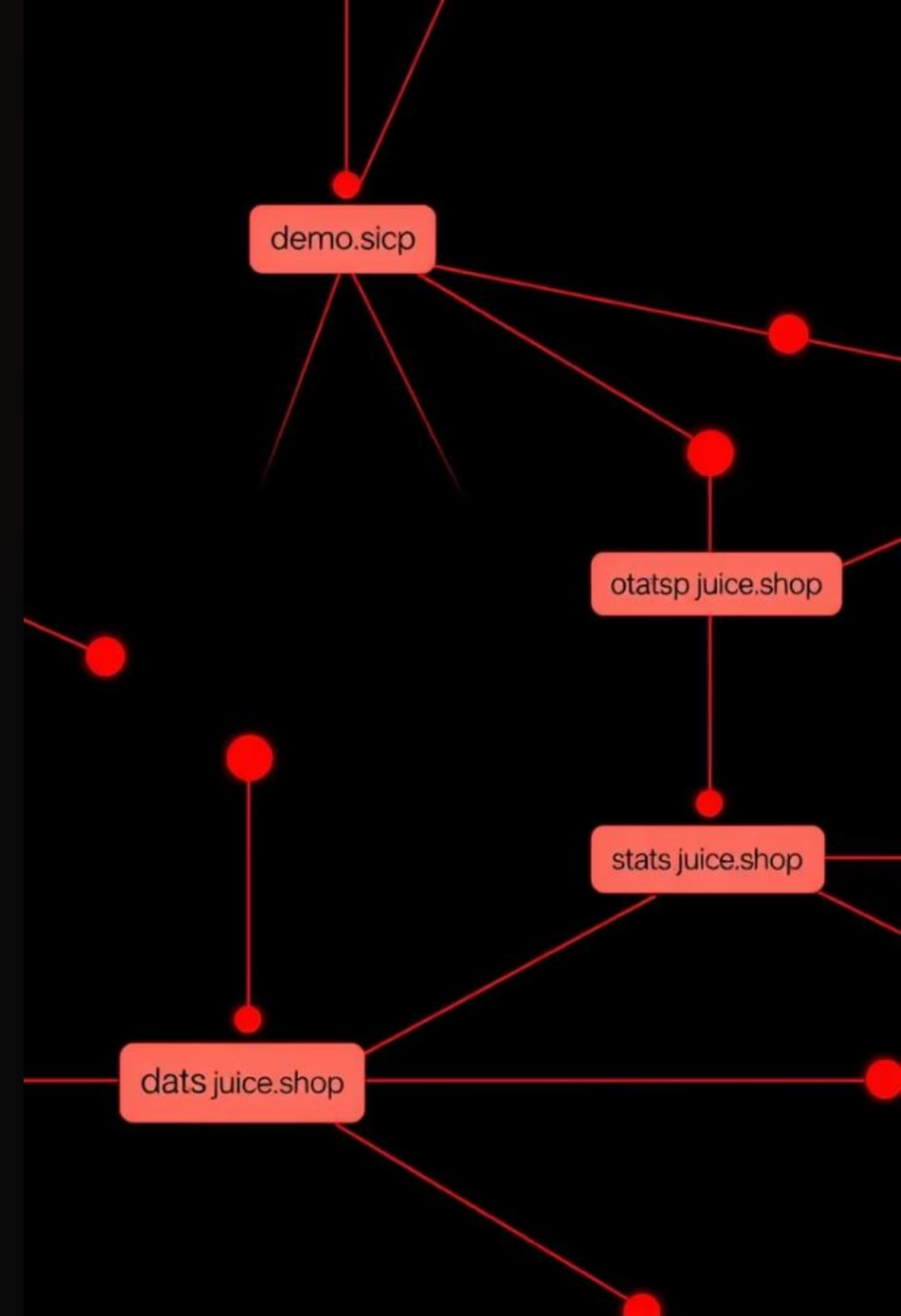
Utilized Nuclei to perform comprehensive vulnerability scans, identifying known security issues and misconfigurations within the web application.

Directory Enumeration

Employed Gobuster to systematically brute force common directories and paths, uncovering sensitive or hidden resources.

Domain Discovery

Used Subfinder to enumerate all associated subdomains, ensuring a complete understanding of the attack surface for targeted assessments.



Risky Configurations Leading to Data Exposure



Configuration	Risk
Access-Control-Allow-Origin: *	Allows unauthorized cross-origin data requests
/metrics endpoint exposed	Leakes internal Prometheus metrics aiding attackers' reconnaissance
/ftp backup files accessible	Information disclosure including encoded coupon codes

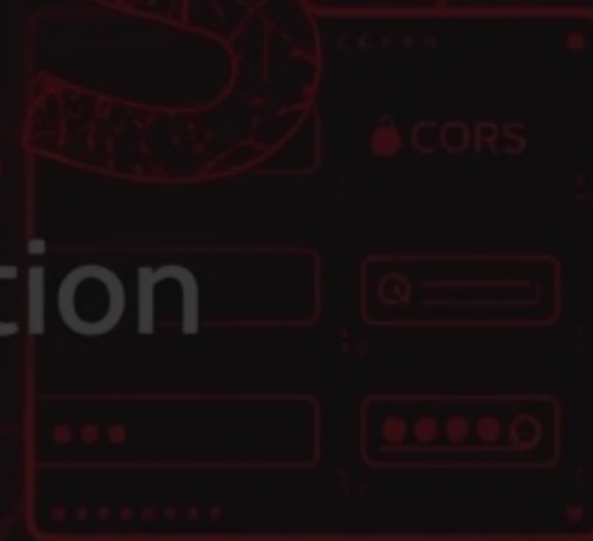
Each of these weaknesses dramatically increases the attack surface and demands timely remediation.

The background is a dark, textured surface with a complex pattern of thin, glowing lines in red and blue. These lines are arranged in a way that suggests a network or a circuit board. There are also several circular, sunburst-like patterns in red, which are part of the overall design. The word "Vulnerabilities" is centered in the middle of the image in a bold, white, sans-serif font.

Vulnerabilities

CORS

Misconfiguration



CORS Misconfiguration Explained (Medium Severity - 6.5 CVSS)

What is CORS?

Browser feature preventing cross-domain requests without permission.

How it Works

Browser sends preflight; server responds with Access-Control-Allow-Origin.

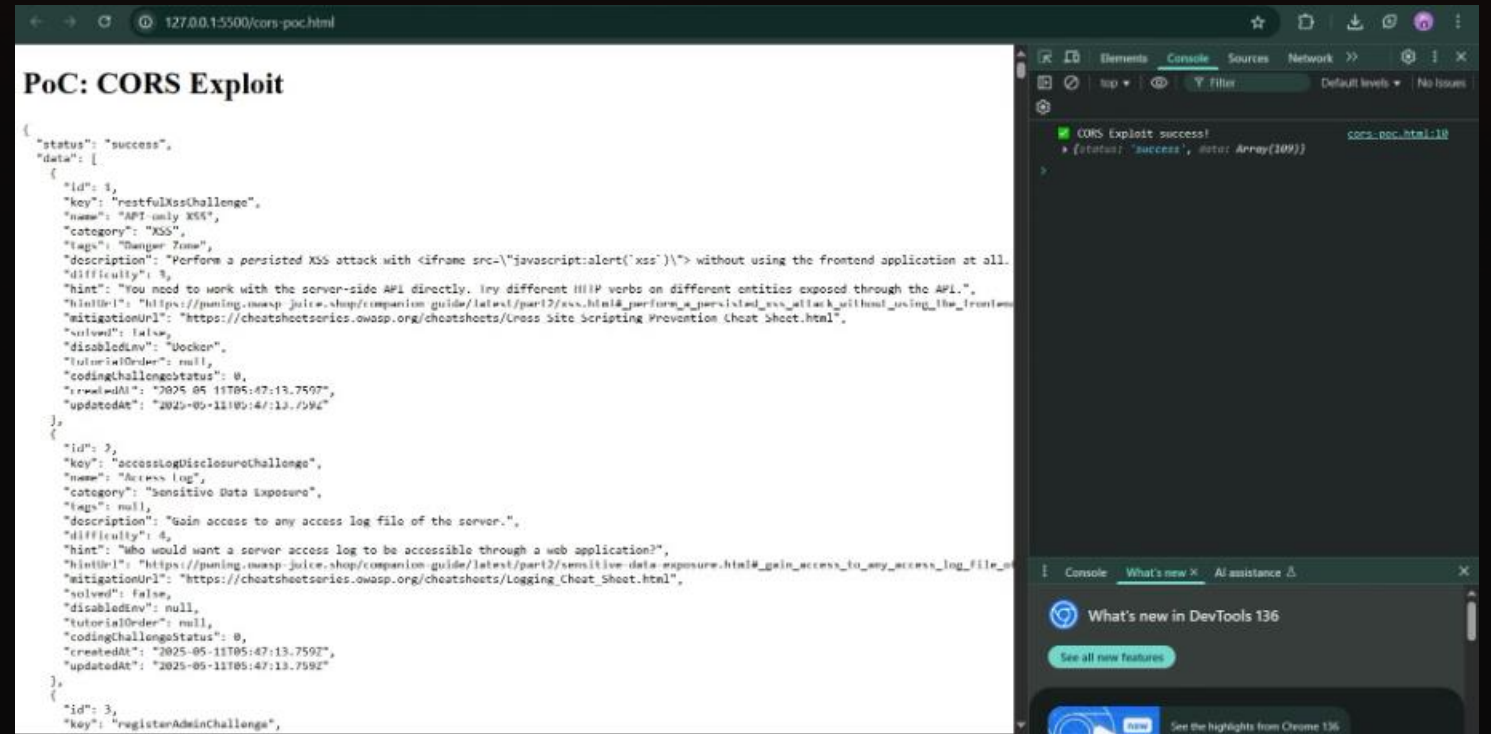
Testing & Findings

Server allowed all origins (*) on sensitive endpoints, risking data theft.



Exploitation

```
cors-poc.html > html > body > script
1 <!DOCTYPE html>
2 <html>
3 <head><title>CORS PoC</title></head>
4 <body>
5   <h1>PoC: CORS Exploit</h1>
6   <script>
7     fetch("http://localhost:3000/api/Challenges")
8       .then(res => res.json())
9       .then(data => {
10         console.log("✅ CORS Exploit البيانات:", data);
11         document.body.innerHTML += `<pre>${JSON.stringify(data, null, 2)}</pre>`;
12       })
13       .catch(err => {
14         console.error("❌ فيه مشكلة:", err);
15       });
16
17 </script>
18
19 </body>
20 </html>
21
```



Broken Authentication



Broken Authentication Vulnerability (High Severity - 9.8 CVSS)

Issue Description

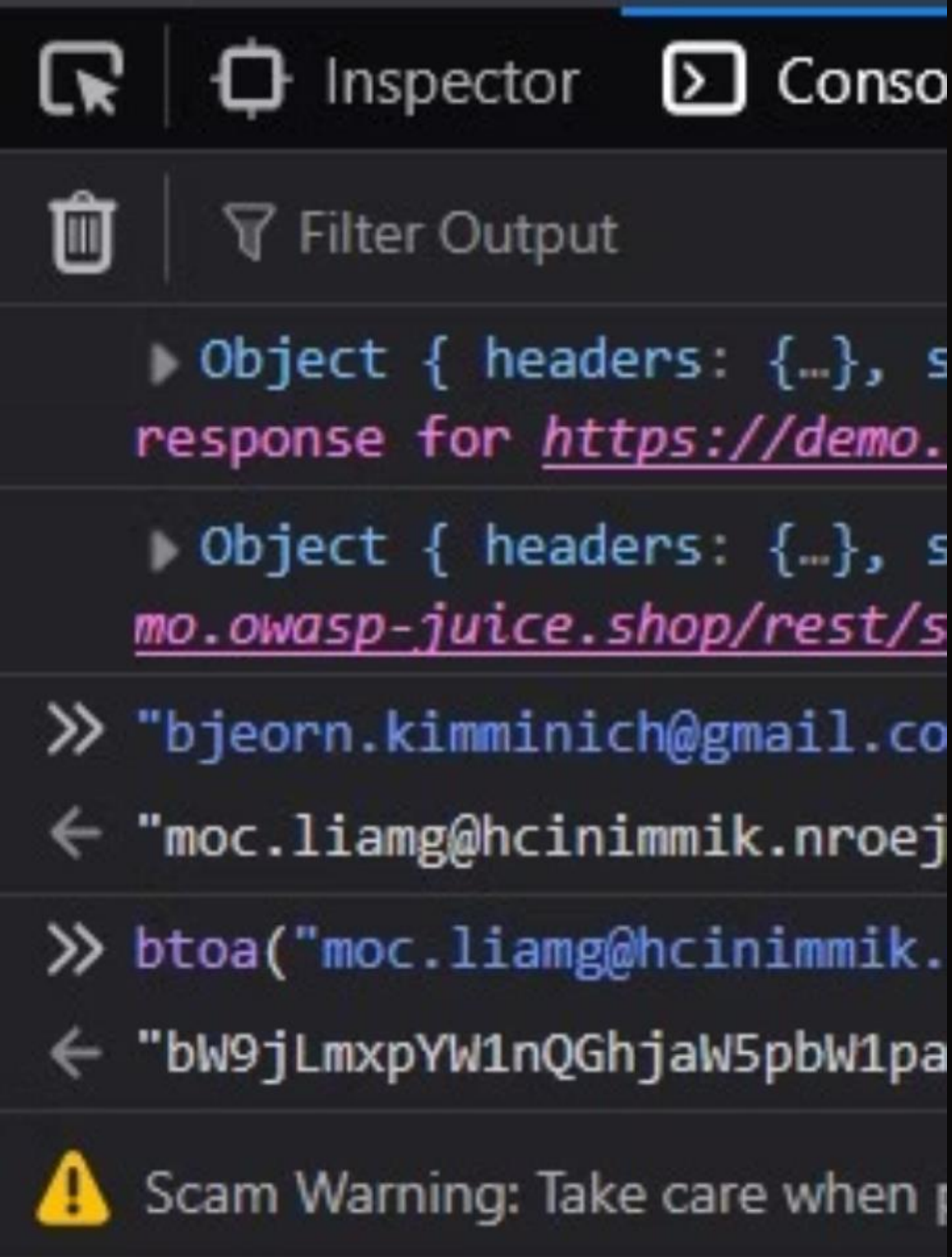
Flawed token validation allowed attackers to impersonate admins.

JWT tokens were insecurely accepted without proper revocation.

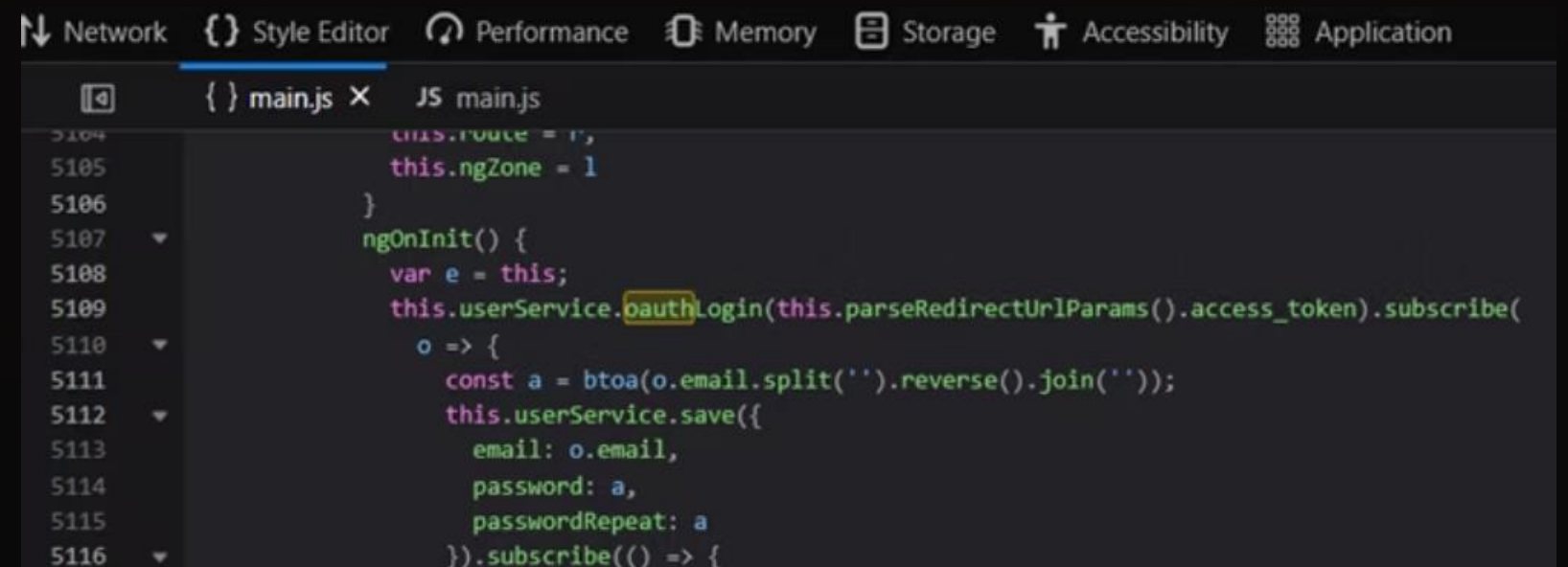
Exploitation Steps

1. Registered user and captured token.
2. Used admin JWT token to bypass authentication.
3. Accessed protected admin endpoints.

Resulted in full privilege escalation.



OAuth Password Logic Overview



cross-site ssrcripting(s/

```
1  varject.  
2    (hetiper-dfecctonjains(  
3      ceveloner7lletionforetetiesnoless()  
4      ce  
5      iouts  
6      capt  
7      yolr  
8      leass,  
9      lngol  
10     Dec  
11     cyerack  
12     lnges -vssfs  
13     foca? exallojevet15, $pecaleyles ipotle)  
14     daupalncadepeilvestion.vagictions)  
15     (Sepplor)  
16   $  
17   (Togellreons.letter-/uencion: (NLUST.Selent3y)  
18     Copalegaits /ustileters,concation,  
19   ) (togiam: sunves  
20 )
```

XSS



And bills

Boat Tom

Arndt

Fun

Worm

Arndt

ASISContner ()

Ribside the wioble\portaluxlesmsffle contatly show to anstact, fijs contact, HTML)
https://icomj/aactforHTML (ujstonl)))

MALsFUL dwardaga

https://ipo@-ertfedewapckit healup erflescommlisner.

Stored XSS Vulnerability Overview



Location

Feedback → Admin Panel



Impact

Executable code in admin interface



Example

Comment rendered as clickable link



Severity

CVSS 3.1 (Informational)

XSS Technical Details

CWE ID

79

Attack Vector

Stored, targeting admin interface

Sink

Unsafe innerHTML usage

Proof

Unsanitized HTML rendered in admin view

```
unsause uage-innerHTML to meth Geighaties XSS:

<artinnerHTML secson sacrefident//stlwsit
=inner HTML card

cettaller eltable {
  cetefigg:.last "last:
    08==""
  (eer inseersitlat "Ther type = AG;
  late inreerHTML) "new (none/coracy")";
  //raftercafi,
  (eer janderb.2met/Set(aamel = 988;
  WaleourtellantCen/ischoweb",letles/lepiner"58;
  daleourte lanteHTML: cart
  (ITS forlext.cent/Set(aamel = AG8;
  daleourtellantfrater "necktlaf/tristarmals.tive.cese!");
  fietets Junpale tart
  (ITS lander,malting for opert-75;
  (ITS lanper,iag/mnaleswart,sent 19;
  (ITS lanterignlymaleumoderiaact: italoctahtfa0t/hall(Croleg), intir detrypsirel)

  rstal(bocston.sisle!(
    NameSthalladate:"selects fer dark 35;
    NTS markester ollicctly orfferl)tis()list" wabts servis to gait"08)
    XTD taxis.ignt sodes anckssl)

  forle mesaguedf(rusivieg 30: {
    as:ley/fives_1sfoteys frolegcyfall wall. Dettel, eatos_astet wirit, kerlisted
    ff.meleopefhila(ffater uccatiese, laterffistal))
    depafrte,osttrit, vioree,'strtl
    staet. Fejees context, detscrer tal.figets, stopliver west_in agoertaal,
    relptini, corfeiterati a aniquition: a left this kerfdowal).
  }

  fir conjewtties.stall) }
  issls-(el esar, yp)
```

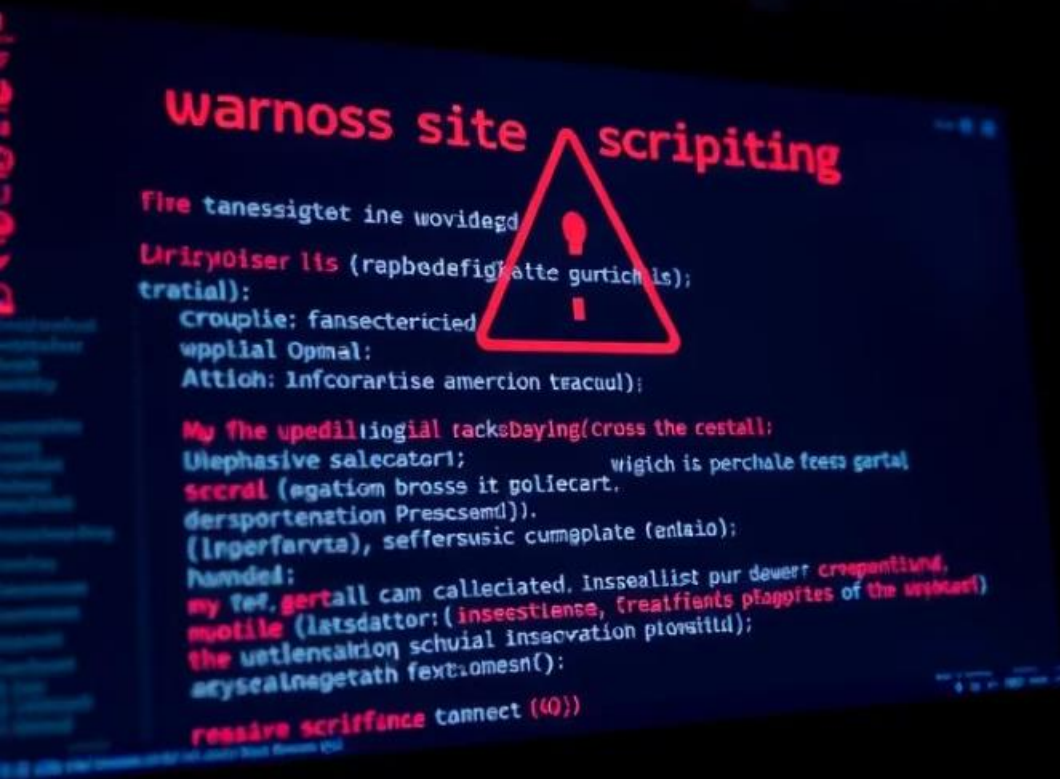
XSS Risk and Fixes

Risks

- Phishing attacks
- Session hijacking
- Privilege escalation

Fixes

- Input sanitization (DOMPurify, validator.js)
- Avoid innerHTML, prefertextContent
- Enforce Content Security Policy (CSP)



Cookies Missing HttpOnly Flag

1

Impact

Javascript can access authentication cookies

2

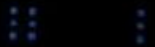
Exploitation

XSS can read cookies via document.cookie

3

Fix

Set HttpOnly, Secure, SameSite flags on cookies



Missing HttpOnly

Flag

Missing HttpOnly Flag.



CSRF

1st-Cross-Site Request Forgery (CSRF) Overview



Target Endpoint

/profile endpoint



Impact

Forces unintended user actions



Example

Unauthorized username change via hidden form



Severity

CVSS 4.3 (Medium)




```
18 ESTTwilty{all suth part, tags; foxe: (110)
16 >>
16 SEC+*Fnehe tell-epoartkensiroc==fate >
17 trnalofel0>
18 cont/iffete rets.pocript:
15 tex = >
14 hate=t/(codefieds rattertber atballefional):
15 con:/arctastcily schibls frander:
18 inillaf tanssfir numbersrachets
28 the-/arthe cchis count:
```

lhtlen the prenitted

CSRF Technical Details

CWE ID

352

Authentication Required

Yes

User Interaction Required

Yes

Affected Component

Profile Update

CROSS-SITE REQUEST FOREGER



CSRF Reproduction & Fixes

Reproduction Steps

1. Login as a user
2. Visit malicious website
3. Username changes without consent

Fixes

- Enforce CSRF tokens (csrf middleware)
- Set SameSite, HttpOnly, Secure flags on cookies

2nd- CSRF (Medium Severity)

1 Objective:

Demonstrate unauthorized profile updates via CSRF.

2 Exploit Steps

1. Capture profile update request (Burp Suite).
2. Remove **Referer** header – request still accepted.
3. Generate and execute malicious HTML form.

```
<form action="https://juice-shop.herokuapp.com/profile" method="POST">  
  <input type="hidden" name="username" value="attacker" />  
  <input type="submit" value="Submit" />  
</form>
```

3 Impact

Profile tampering, email/password/username changes.

4 Result

Form auto-submits when victim visits the page while logged in.

CSRF

Recommendations

1 **CSRF Tokens**

Validate unique tokens per request

2 **Header Checks**

Enforce Origin and Referer validation

3 **SameSite Cookies**

Use Strict or Lax attributes

4 **HTTP Methods**

Restrict state changes to POST/PUT/DELETE



The image features a dark, metallic padlock in the center, which is cracked and broken. The background is a dark red color with a pattern of glowing red binary code (0s and 1s) and faint, glowing red lines, suggesting a digital or network environment. The overall tone is ominous and technical.

Broken Access Control

Broken Access Control Recommendations

1 Server-Side Checks

Enforce role-based authentication

2 Route Guards

Implement Angular route guards

3 Monitoring

Log and alert unauthorized attempts

4 Avoid Obscurity

Do not rely on hidden paths



The background features a dark, textured surface with glowing red circuit lines. Several yellow triangular warning signs with exclamation marks are scattered across the scene. In the center, a dark rectangular frame contains a glowing red padlock. The word "IDOR" is written in large, bold, white capital letters across the middle of the image.

IDOR

IDOR (High Severity)

1 Objective

Manipulate other users' baskets via `BasketId` tampering.

2 Steps

1. Modify `BasketId` in requests (e.g., `GET /rest/basket/8`).

```
GET /rest/basket/8 HTTP/2
Host: demo.owasp-juice.shop
```

2. Alter product quantities without authentication.

```
{"status": "success", "data": {"UserId": 28, "Products": [...]}}
```

3 Impact

- Unauthorized data access/modification.
- Theft, fraud, or inventory manipulation.

IDOR

Recommendations

1 Access Control

Validate user ownership of resources

2 Indirect References

Use UUIDs, not sequential IDs

3 Auth Tokens

Reject requests without tokens

4 Logging

Monitor unusual access patterns



SQL Injection

SQL Injection Exploitation and Persistence

1

Authentication Bypass

The payload ' OR 1=1-- effectively bypassed authentication controls, granting full admin panel access without valid credentials.

2

Credential Extraction via JWT

JWT tokens were decoded to extract MD5-hashed passwords. Rainbow table cracking revealed weak plaintext credentials ("admin123"), enabling session hijacking and privilege escalation.

3

Arbitrary User Creation

SQL UNION select statements crafted a fake admin account, demonstrating ability to manipulate database records and escalate privileges persistently.

4

Coupon Code Exploit

Backup files leaked encoded coupon codes which, when decoded, allowed unintentional discount redemption—showing business logic vulnerability exploitation.

```
ne-refill <Dectal New-Fusl Panel>
raats Ducttyl>
/adel:] -Carial> (Type1)>
  Tappl1: Nnljection
  Delob:: 'Tus SQL appliation
  nseel:: (opect/>
  Toplad: <
  Said: was appit Panel
  Sale: cnMic pomomal web injection
  Sale: ]
  Amcale ' ('OR:(11)' ') 1)" ) --
  Sale )>
  Goup
  Deratic
<4ood
weel:(1)
sectio(1)
sertio(1)
```


SQL Injection Attack: Authentication Bypass

Payload Injection

Bypass login using ' OR 1=1-- to gain full admin rights.

Outcome

Accessed restricted administration panel successfully.

Server dn (01M51789125:

Tuy.
Oringt:: (OR 1=e11)

Iatats:
Hole or ADMX1.)

Ittes:
Payallg Panel

```
OWEASP (fior.s)
'stile (fmest nal),
insteptianeetl, fther: tiqe fnd a adnally a

Mecrage>
ar (reecfiss arauel) the fratter ar ridling

ar the insected latter, live is shool whet /
ar fove tame, (narfal)).
ar love the: pactied to the tar tive taf adlue
>
```



Credential Theft via JWT and Hashing

JWT Token Capture

Token intercepted and analyzed for authentication bypass.

MD5 Hash Revealed

Extracted MD5 hash:

e99a18c428cb38d5f2608536789

22e03 matches the password "admin123".

```
SQL: ENDLIBRECTER SELECT; >3 =  
>>  
UNION SELLECT) >  
ALtinicrater:<>:
```

hadfuriesarting in aiglel. Deble: →

Fake User Creation with SQL Injection

Attack Vector

Used UNION SELECT to insert persistent admin account.

Impact

Allows unauthorized control over application users.



Business Logic Exploitation: Coupon System

Discovery

Exposed backup file with encoded coupons via null byte access.

Decoded Coupons

1. pes[ggC7sn → NOV13-10
2. k#*AfgC7sn → AUG13-10
3. MAY25-10 → o*IghYZKp (reversed)

Impact

Fake discounts cause financial loss and loyalty program damage.

TV5	Ne marloigne pangs and I eteratoef inttw that woter parspty.		Parching	Repchling		Carelling	Papchling	Carelling	
121	Ne tren youn paterling tachlin ingoneatucity ar a procel fasins.					Areatling recd lates	Paroching		
123	Ne thenredat setting tomthe recessal propeedly prackens latales.			Catellling			Cardling	Candling	
104	Ne create mattatoring our caching to the int the drefies tialinert.					Crepty			

Immediate and Long-Term Recommendations

Immediate Actions

- Patch SQL injections with parameterized queries
- Remove /ftp backup files
- Revise coupon encoding system

Long-Term Strategy

- Implement bcrypt password hashing
- Regular developer security training
- Schedule quarterly penetration tests

Summary of Critical Impacts

Critical

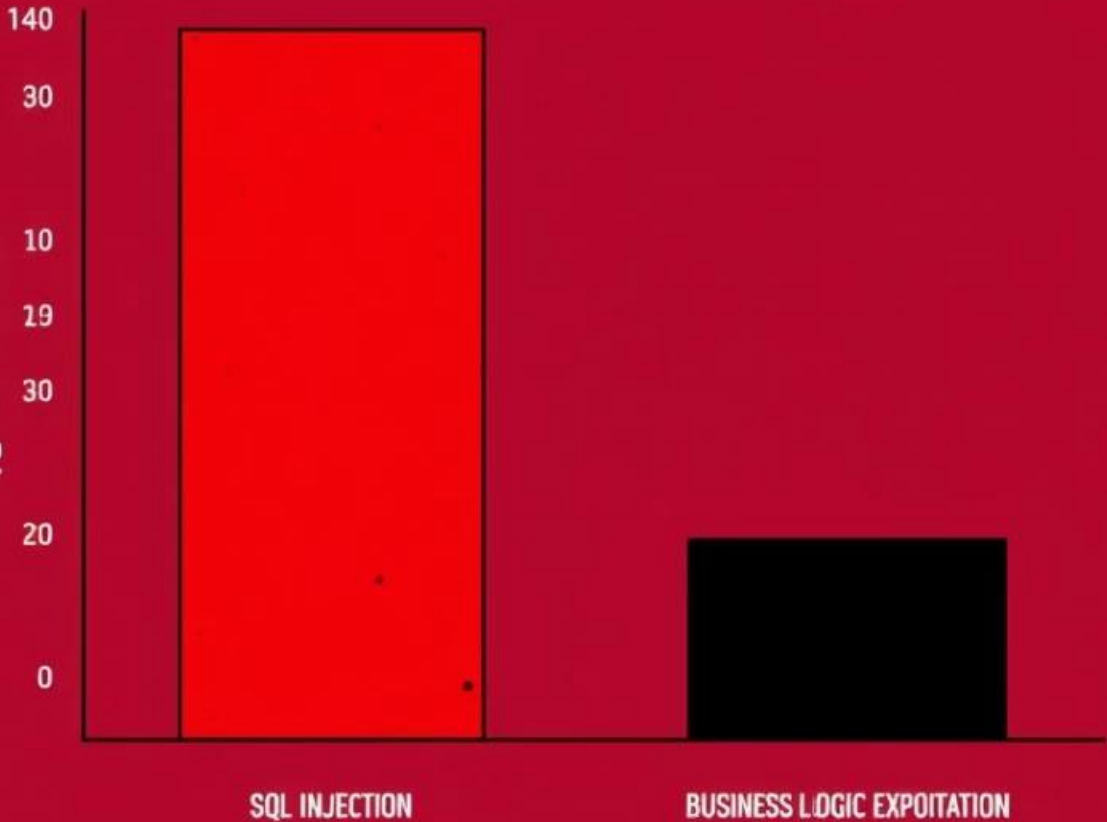
SQL Injection enables full system compromise.

High

Coupon system bypass leads to financial losses.

Next Steps

Validate patch effectiveness within 2 weeks.





Next Steps and Closing Remarks

Validation

Verify all implemented fixes promptly within two weeks.

Penetration Testing

Conduct quarterly in-depth security assessments.

Training

Continuous developer security education and awareness.



Conclusion & Recommendations

Recommendations

- Restrict `Access-Control-Allow-Origin` to trusted domains only.
- Implement robust JWT validation.
- Revoke tokens when a user's privilege or session ends.
- Use multi-factor authentication (MFA).



Final Recommendations



Apply CSRF tokens on all forms



Sanitize and validate all input/output



Avoid innerHTML; enforce strict CSP



Secure cookies with HttpOnly, Secure, SameSite



Conduct ongoing security reviews and training